



ESPRIT School of Business

Honoris United Universities

Master 1 - Business Analytics (M1 BA)

Projet Intégré

Analyse et Prédiction du Churn Client Bancaire

De l'exploration des données au déploiement d'un modèle prédictif :
un pipeline ETL, un entrepôt de données, des tableaux de bord Power BI
et une application web de scoring

Équipe HAFAS

Feres Messedi · Eya Bhrouni · Hamza Bouguerra
Aïcha Soltani · Sahar Khalfellaoui

Tuteur du projet
Aymen Ben Brik

Juillet 2026

Résumé exécutif

Ce projet construit, de bout en bout, une chaîne complète d'analyse du churn (attrition) client pour une banque, à partir d'un extrait anonymisé de 528 883 lignes et 34 colonnes. La démarche suit six étapes : exploration des données, ETL vers un entrepôt en étoile PostgreSQL (6 tables de dimension, 445 803 événements de fait), tableaux de bord Power BI, modélisation supervisée (prédiction du churn), modélisation non supervisée (segmentation et détection d'anomalies), et une application web Streamlit de scoring individuel.

Le pipeline ETL a été validé par un script de cohérence automatisé (32 tests) et a permis de découvrir et corriger 7 anomalies réelles dans les données sources (règles de gestion incomplètes, pollution de dates d'extraction, valeurs aberrantes). Six modèles de classification (testés avec et sans SMOTE, soit dix configurations) ont été comparés au grain client (319 129 clients, taux de churn 36%) ; le modèle retenu, XGBoost avec SMOTE, est sélectionné sur la base du **recall** (84,92%, le meilleur parmi les modèles à précision exploitable), avec un F1-score de 88,24% et une AUC-ROC de 94,64%, après qu'une fuite de données (**acct_balance**, manquante pour 100% des clients churnés) a été détectée et exclue en cours de construction. Une analyse complémentaire non supervisée, au même grain client, identifie quatre segments par K-means et 3 192 clients anormaux par Isolation Forest.

Les résultats sont restitués à travers trois tableaux de bord Power BI (vue d'ensemble, analyse du profil client, analyse du risque et de la valeur en risque) et une application Streamlit permettant un scoring en temps réel. Ce rapport documente également les limites assumées du projet et les pistes d'amélioration identifiées pour chaque étape.

Mots-clés : churn bancaire, entrepôt de données, modélisation dimensionnelle, ETL, Power BI, apprentissage supervisé, XGBoost, segmentation client, détection d'anomalies.

Abstract

This project builds an end-to-end customer churn analysis pipeline for a bank, based on an anonymized extract of 528,883 rows and 34 columns. The workflow covers six stages : data exploration, ETL into a PostgreSQL star schema (6 dimension tables, 445,803 fact events), Power BI dashboards, supervised modeling (churn prediction), unsupervised modeling (segmentation and anomaly detection), and a Streamlit web application for individual scoring.

The ETL pipeline was validated through an automated coherence-testing script (32 tests) and surfaced 7 genuine data-quality issues that were investigated and corrected. Six classification models (tested with and without SMOTE, ten configurations in total) were compared at client grain (319,129 clients, 36% churn rate); the selected model, XGBoost with SMOTE, is chosen on the basis of **recall** (84.92%, the best among models with usable precision), reaching an F1-score of 88.24% and a ROC-AUC of 94.64%, after a data-leakage issue (**acct_balance**, missing for 100% of churned clients) was detected and excluded during development. A complementary unsupervised analysis, at the same client grain, identifies four K-means segments and 3,192 anomalous clients via Isolation Forest.

Results are presented through three Power BI dashboards and a Streamlit application enabling real-time individual scoring. This report also documents the project's known limitations and the improvement paths identified at each stage.

Keywords : banking churn, data warehouse, dimensional modeling, ETL, Power BI, supervised learning, XGBoost, customer segmentation, anomaly detection.

Table des matières

Résumé exécutif	1
Abstract	2
1 Introduction et contexte	5
1.1 Contexte du projet	5
1.2 Objectifs	5
1.3 Équipe et organisation	5
1.4 Dépôt de code et gestion de versions	6
1.5 Plan du rapport	7
2 Description et exploration des données	8
2.1 Origine et anonymisation	8
2.2 Volumétrie et grain	8
2.3 Définition retenue du churn	8
2.4 Qualité des données - constats de l'exploration	8
3 Architecture de la solution	10
3.1 Vue d'ensemble du pipeline	10
3.2 Choix technologiques	10
3.3 Schéma en étoile	10
4 ETL et modélisation dimensionnelle	12
4.1 Pipeline ETL	12
4.1.1 Anomalies réelles découvertes et corrigées	12
4.1.2 Vérification de la nullité structurelle de <code>date_key</code>	12
4.2 Entrepôt de données PostgreSQL	13
4.3 Comparaison Talend vs Python	13
5 Analyses BI et tableaux de bord Power BI	15
5.1 Vue d'ensemble	15
5.2 Analyse du profil client	15
5.3 Analyse produits et risque financier	16
5.4 Décomposition de la Value-at-Risk (VaR)	17
5.5 Limites de cette partie	17
6 Modélisation supervisée - prédiction du churn	18
6.1 Grain et périmètre	18
6.2 Sélection des variables	18
6.3 Fuite de données détectée et corrigée	18
6.4 Prétraitement	19
6.5 Modèles comparés et résultats	19
6.6 Critère de sélection : pourquoi le recall prime sur ce problème	19
6.7 Limitations assumées	20
7 Analyse non supervisée - segmentation et anomalies	21
7.1 Motivation et grain	21

7.2	Analyse en composantes principales (ACP)	21
7.3	Segmentation par K-means	21
7.4	Détection d'anomalies par Isolation Forest	22
7.5	Usage envisagé	22
8	Interface web et déploiement	23
8.1	Application Streamlit	23
8.2	Déploiement	23
8.3	Point de vigilance	23
9	Limites et perspectives	24
9.1	Limites connues	24
9.2	Perspectives	24
10	Conclusion	25
	Bibliographie	26
A	Annexes	27
A.1	Glossaire	27
A.2	Structure du dépôt de code	27
A.3	Extrait de code - contrôle de nullité structurelle de <code>date_key</code>	27

Chapitre 1

Introduction et contexte

1.1 Contexte du projet

Dans le secteur bancaire, la perte d'un client (*churn*) représente un coût direct (perte de revenus récurrents) et indirect (coût d'acquisition d'un nouveau client, souvent estimé à plusieurs fois le coût de rétention d'un client existant). Anticiper le churn avant qu'il ne se produise permet à une banque de cibler ses actions de rétention (offres personnalisées, contact proactif d'un conseiller) vers les clients à risque, plutôt que d'agir de manière non ciblée.

Ce projet a été réalisé dans le cadre du Projet Intégré du programme Master 1 Business Analytics de l'ESPRIT School of Business, sous la direction de M. Aymen Ben Brik ([Ben Brik, 2026](#)). L'objectif pédagogique est de couvrir, en équipe, l'ensemble de la chaîne de valeur de la donnée : de l'exploration d'un jeu de données brut jusqu'à la restitution d'un modèle prédictif via une interface utilisateur, en passant par la modélisation dimensionnelle et la visualisation décisionnelle.

1.2 Objectifs

Les objectifs du projet, tels que définis dans le cahier des charges fourni par l'encadrant, sont les suivants :

1. Explorer et comprendre un jeu de données bancaire anonymisé (528 883 lignes, 34 colonnes) et documenter sa qualité.
2. Concevoir et implémenter un pipeline ETL reproductible, chargeant les données nettoyées dans un entrepôt de données relationnel structuré en schéma en étoile.
3. Restituer les indicateurs clés de churn à travers des tableaux de bord Power BI, destinés à un public à la fois métier et technique.
4. Construire et comparer plusieurs modèles de classification supervisée pour prédire le churn au niveau compte, et documenter la démarche de validation (fuites de données, métriques, interprétabilité).
5. Compléter l'analyse supervisée par une démarche non supervisée (segmentation client, détection d'anomalies) pour éclairer des angles que la seule prédiction binaire ne couvre pas.
6. Déployer le modèle retenu dans une application web interactive, permettant un scoring individuel en temps réel.

1.3 Équipe et organisation

Le projet a été mené par l'équipe HAFAS, composée de cinq étudiants : Feres Messedi, Eya Bhrouni, Hamza Bouguerra, Aïcha Soltani et Sahar Khalfellaoui. Le travail a été organisé par lots correspondant aux grandes étapes du pipeline (exploration, ETL, entrepôt de données, Power BI, machine learning, datamining, application web), avec un suivi de version Git partagé et une revue

de code croisée entre membres avant intégration de chaque module.

1.4 Dépôt de code et gestion de versions

L'ensemble du code, des notebooks et de la documentation du projet est versionné sur GitHub :

- **Dépôt du projet** : https://github.com/Hmz931/ProjetIntegre_Churn_HAFAS
- **Dépôt modèle fourni par l'encadrant** : <https://github.com/Aymenbenbrik/ProjetIntegre>, utilisé comme point de départ pour la structure de dossiers et la documentation (00_documentation/).

Le développement s'est appuyé sur plusieurs branches de travail : **main** (branche stable, intégrant les livrables validés), **ETL_EyaFares** (pipeline ETL et modélisation dimensionnelle, porté principalement par Eya Bhrouni et Feres Messedi) et **AishaHamzaSahar** (modélisation et restitution, porté principalement par Aïcha Soltani, Hamza Bouguerra et Sahar Khalfellaoui), fusionnées dans **main** après revue de code croisée entre membres de l'équipe.

Cette répartition par branche reflète une organisation initiale du travail, non une distribution stricte et cloisonnée des rôles : chaque membre est intervenu ponctuellement sur des lots portés par d'autres (revue de code, correction de bug, complément de documentation), dans une logique transversale plutôt que par silos étanches. Cette souplesse a notamment permis de détecter et corriger plusieurs anomalies croisées entre étapes (par exemple la fuite de données décrite à la section 6.3, identifiée grâce à un regard croisé entre le sous-groupe ETL et le sous-groupe modélisation).

Le suivi opérationnel des tâches (sprints, avancement, attribution) a été organisé sur un tableau Trello partagé : <https://trello.com/b/fRd3Q132/projet-pi-esb-1ba2>, structuré en huit listes correspondant aux étapes du pipeline (EDA, ETL, Data Warehouse, Machine Learning, Power BI, Interface Web, Livrables), avec des étiquettes d'estimation de charge et un assignataire par carte.

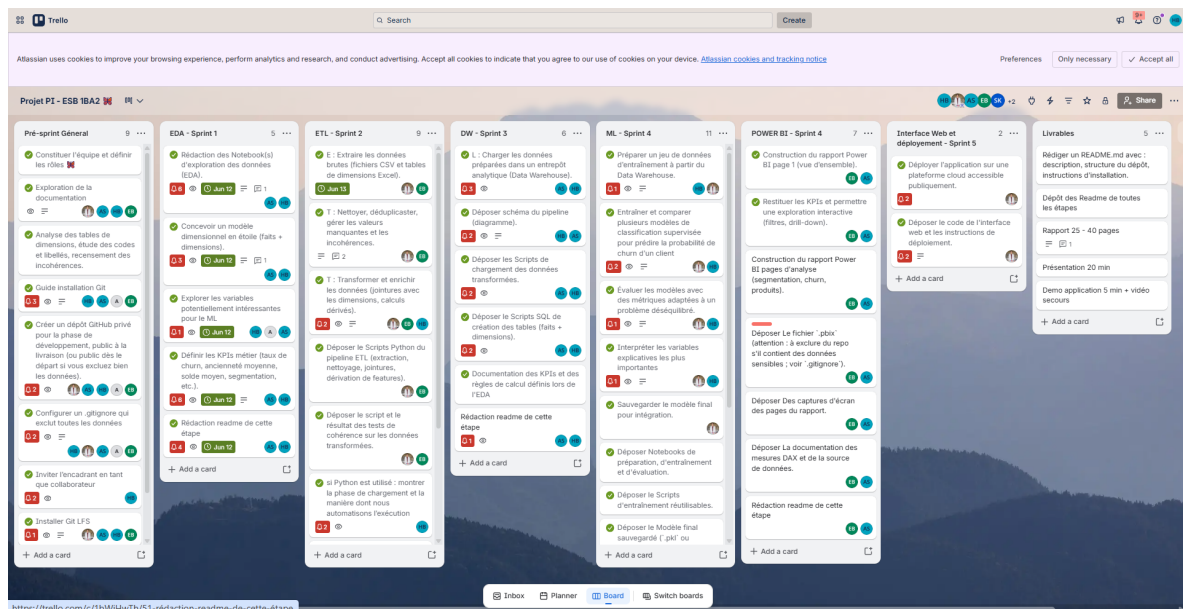


Figure 1.1 : Tableau Trello de suivi des sprints du projet

1.5 Plan du rapport

Ce rapport suit l'ordre du pipeline de traitement de la donnée : description et exploration des données (chapitre 2), architecture de la solution (chapitre 3), ETL et modélisation dimensionnelle (chapitre 4), tableaux de bord Power BI (chapitre 5), modélisation supervisée (chapitre 6), analyse non supervisée (chapitre 7), application web (chapitre 8), limites et perspectives (chapitre 9), puis conclusion (chapitre 10).

Chapitre 2

Description et exploration des données

2.1 Origine et anonymisation

Le jeu de données source provient d'un extrait réel d'un système d'information bancaire, anonymisé avant transmission à l'équipe : tous les codes internes propriétaires ont été renommés, les identifiants clients et comptes remplacés par des séquences (C0000001, A0000001...), les agences renumérotées aléatoirement, et la date de naissance réduite à l'année. Aucune information directement identifiante (nom, email, téléphone, IBAN) n'est présente. Le script d'anonymisation (`anonymize.py`) est fourni avec le dépôt pour traçabilité.

2.2 Volumétrie et grain

Le fichier source (`data_churn.txt`) contient **528 883 lignes et 34 colonnes**, au grain événement (une ligne par produit rattaché à un compte). Huit fichiers de référentiel Excel (`dim_*.xlsx`) complètent le jeu de données : catégories de compte, devises, motifs de clôture, secteurs d'activité, segments cibles, transactions.

Après nettoyage (chapitre 4), le jeu de données se réduit à **445 803 événements**, correspondant à **410 587 comptes uniques**, **319 129 clients uniques** et **141 agences**.

2.3 Définition retenue du churn

Le churn est défini au grain **compte** : `churn = 1` si `ACCOUNT_STATUS == "Closed"`. Cette définition est propagée au grain événement (chaque ligne liée à un compte fermé hérite de `churn=1`), ce qui produit deux taux de churn selon le grain d'observation :

- **36,1%** au grain compte (410 587 comptes) - le taux de référence utilisé pour la modélisation supervisée (chapitre 6).
- **41,2%** au grain événement (445 803 lignes) - plus élevé car les comptes fermés ont, en moyenne, un nombre de produits associés légèrement différent des comptes actifs.

Pour l'analyse non supervisée au grain client (chapitre 7), une définition alternative a été retenue : `churn = 1` si et seulement si *tous* les comptes du client sont fermés (agrégation par `min` plutôt que par `max`), ce qui évite de considérer comme churné un client ayant simplement fermé un compte secondaire tout en gardant une relation active avec la banque.

2.4 Qualité des données - constats de l'exploration

L'exploration initiale (notebook `01_exploration.ipynb`) a mis en évidence plusieurs problèmes de qualité, tous documentés et traités dans l'étape ETL :

- **38 640 doublons stricts** (lignes identiques sur toutes les colonnes), supprimés.

- **44 440 lignes sans numéro de compte (ACCOUNT_NO)** (8,4% du fichier dédoublonné), jugées inexploitable et supprimées.
- Des **dates aberrantes** (années de naissance antérieures à 1900, dates d'ouverture postérieures aux dates de clôture) - retraité au cas par cas (voir section [4.1.1](#)).
- Des **valeurs extrêmes de salaire** (jusqu'à 25 millions de TND), incompatibles avec les profils clients concernés - confirmées comme erreurs de saisie et neutralisées.
- Des **valeurs manquantes structurelles** sur ACCT_BALANCE et SALARY, volontairement conservées (non imposées) faute d'une méthode d'imputation fiable au vu de l'asymétrie des distributions.

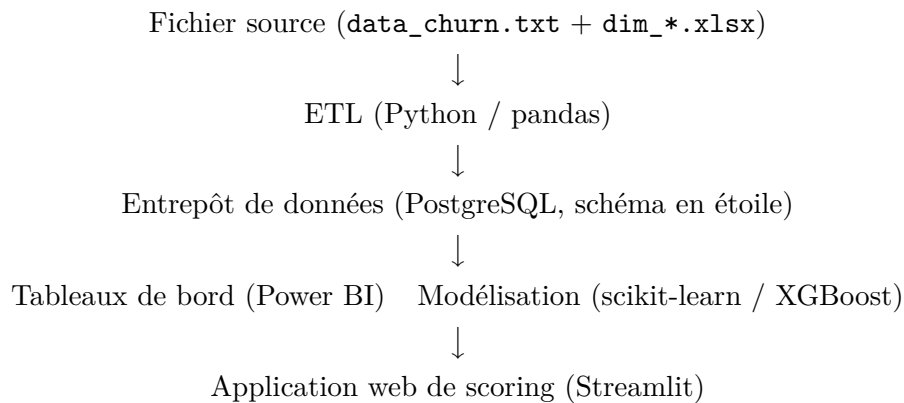
Ces constats ont directement guidé les règles de nettoyage implémentées dans `01_etl/etl_pipeline/clean.py`, détaillées au chapitre [4](#).

Chapitre 3

Architecture de la solution

3.1 Vue d'ensemble du pipeline

La solution est organisée en six étapes séquentielles, chacune consommant la sortie de la précédente :



Chaque étape est indépendante et peut être réexécutée isolément : le pipeline ETL expose une fonction `run_pipeline()` réutilisable en mémoire (sans nécessiter PostgreSQL), sur laquelle s'appuient à la fois les notebooks de machine learning et de datamining lorsqu'aucune base n'est accessible.

3.2 Choix technologiques

Composant	Technologie retenue et justification
Traitement des données	Python + pandas - traitement vectorisé, code versionné et testable (voir comparaison Talend vs Python, section 4.3)
Entrepôt de données	PostgreSQL - SGBD relationnel open source, contraintes d'intégrité référentielle natives
Visualisation décisionnelle	Power BI Desktop - imposé par le cahier des charges
Machine learning	scikit-learn (Pedregosa et al., 2011) + XGBoost (Chen and Guestrin, 2016) + imbalanced-learn (SMOTE, Chawla et al., 2002)
Application web	Streamlit - prototypage rapide d'une interface data science sans développement front-end séparé

3.3 Schéma en étoile

Le modèle dimensionnel retenu suit la méthodologie de Kimball (Kimball and Ross, 2013) : une table de faits centrale (`fact_account_event`, au grain événement `compte×produit`) reliée à six tables de dimension :

Dimension	Contenu
dim_client	Attributs client (âge, statut marital, segment, score KYC...)
dim_account	Attributs compte (catégorie, devise, statut, motif de clôture)
dim_branch	Agence
dim_product	Ligne/groupe de produit, au grain contrat
dim_date	Calendrier (jour, mois, trimestre, année)
dim_closure	Motif de clôture, classifié volontaire/involontaire

Toutes les clés de substitution (*surrogate keys*) sont des chaînes de caractères préfixées et numérotées (CLI_000001, DATE_00001, ACC_0000001...), avec une largeur de zéro-remplissage calculée dynamiquement selon le volume de chaque dimension - ce choix facilite la lecture des clés en base et évite toute ambiguïté avec un identifiant métier numérique.

Chapitre 4

ETL et modélisation dimensionnelle

4.1 Pipeline ETL

Le pipeline ETL est implémenté en Python (module `01_etl/etl_pipeline/`), orchestré en cinq étapes séquentielles : **extract** (lecture du CSV et des référentiels Excel), **clean** (règles de nettoyage), **dimensions** (construction des tables de dimension et de leurs clés), **fact** (construction de la table de faits), **load** (chargement PostgreSQL via SQLAlchemy). Un script de test de cohérence indépendant (`test_coherece_donnees.py`, 32 tests répartis en 5 catégories - complétude, domaine, inter-colonnes, temporel, cardinalité) valide le résultat sans modifier le pipeline lui-même.

4.1.1 Anomalies réelles découvertes et corrigées

Sept anomalies concrètes ont été identifiées en testant le pipeline contre les données réelles, documentées avec leur nombre exact de lignes concernées :

1. **MARITAL_STATUS incohérent pour 25 lignes** (personnes morales) - la règle de correction ne s'appliquait qu'aux valeurs manquantes, laissant passer des valeurs déjà présentes et incohérentes avec un statut de personne morale.
2. **ACCT_OPENING_DATE postérieure à ACCT_CLOSE_DATE** (33 571 lignes) - pollution confirmée par des dates proches de l'extraction du fichier source, neutralisées en valeur manquante plutôt que conservées fausses.
3. **STARTDATE postérieure à MATURITYDATE** (2 575 lignes) - même mécanisme, corrigé de la même manière.
4. **Jointure sur clé nulle** (`dim_Closure_reason`) - une valeur nulle côté référentiel produisait un produit cartésien et des doublons en cascade.
5. **Encodage UTF-8 non forcé** sous Windows/psycopg2 - corrigé en imposant explicitement `client_encoding=utf8`.
6. **Coercition entière imprévue** sur `date_key` lors du chargement - résolue par le passage à des clés textuelles plutôt que numériques (voir ci-dessous).
7. **Dépendance de clé étrangère à la relance** - PostgreSQL refusait de supprimer une dimension tant que la table de faits la référençait encore ; corrigé en supprimant systématiquement la table de faits avant toute opération sur les dimensions.

4.1.2 Vérification de la nullité structurelle de `date_key`

`date_key` est la seule clé étrangère de la table de faits pour laquelle une valeur manquante est normale : elle dérive de `ACCT_OPENING_DATE`, volontairement conservée à vide pour certains comptes. Un contrôle explicite a été ajouté au pipeline : le nombre de `date_key` manquants doit être strictement égal au nombre d'`ACCT_OPENING_DATE` manquantes en amont. Vérifié contre les données réelles : **134 056 lignes (30,1%)**, exactement égal - confirmant une nullité entièrement structu-

relle, sans échec de jointure réel.

4.2 Entrepôt de données PostgreSQL

Le chargement dans PostgreSQL a été validé sur un serveur réel : les 7 tables (6 dimensions + 1 table de faits) se chargent avec succès et toutes les contraintes de clé étrangère s'appliquent sans erreur. La figure 4.1 montre le résultat d'une requête d'inspection sur `dim_account` via pgAdmin, confirmant la structure des clés textuelles et la cohérence des données chargées.

	account_key [PK] text	ACCOUNT_NO text	ACCOUNT_STATUS text	ACCOUNT_CATEGOF double precision	ACCOUNT_TYPE_DESC text	CURRENCY text	ACCT_OPENING_DATE timestamp without time zone	ACCT_CL timestamp
1	ACC_000001	A0365322	Closed	3023	Crédit acquisition logement TEGF6	TND	2019-08-27 00:00:00	2026-01-01
2	ACC_000002	A0373555	Closed	3611	DEPOTS A TERME	TND	[null]	2025-06-01
3	ACC_000003	A0348290	Closed	3017	Crédit rénovation	TND	2023-06-12 00:00:00	2026-01-01
4	ACC_000004	A0257995	Closed	1011	Compte Allocation Touristique TND	TND	2022-05-27 00:00:00	2026-01-01
5	ACC_000005	A0312991	Closed	3611	DEPOTS A TERME	TND	[null]	2025-06-01
6	ACC_000006	A0318636	Closed	3611	DEPOTS A TERME	TND	[null]	2025-06-01
7	ACC_000007	A0312993	Closed	3611	DEPOTS A TERME	TND	[null]	2025-06-01
8	ACC_000008	A0318510	Closed	3611	DEPOTS A TERME	TND	[null]	2025-06-01
9	ACC_000009	A0373623	Closed	3611	DEPOTS A TERME	TND	[null]	2025-06-01
10	ACC_000010	A0356197	Closed	3005	Crédit a la consommation TEGF3	TND	[null]	2023-04-01
11	ACC_000011	A0392911	Closed	8030	Retrait par Carte LELLA	TND	[null]	2023-04-01
12	ACC_000012	A0403856	Closed	3005	Crédit a la consommation TEGF3	TND	[null]	2023-04-01
13	ACC_000013	A0354222	Closed	3005	Crédit a la consommation TEGF3	TND	[null]	2023-04-01

Figure 4.1 : Entrepôt de données PostgreSQL - inspection de `dim_account` via pgAdmin (chargement réel validé, 7/7 tables)

4.3 Comparaison Talend vs Python

Une question récurrente en soutenance concerne le choix d'un pipeline codé en Python plutôt qu'un outil ETL graphique tel que Talend Open Studio (Talend / Qlik, 2026), plus classique dans un cursus Business Analytics. Le tableau 4.1 résume la comparaison, fondée sur l'expérience réelle de ce projet (7 bugs trouvés et corrigés en cours de route).

Critère	Talend Open Studio	Python (pandas)
Courbe d'apprentissage	Interface visuelle, prise en main rapide pour des transformations simples	Nécessite de coder, mais l'équipe le maîtrisait déjà
Traçabilité Git	Jobs stockés en XML propriétaire compilé, diff illisible	Chaque règle métier est un <code>git diff</code> de quelques lignes, lisible en revue de code
Débogage	Débogueur visuel, mais isole mal une transformation précise sur 528 883 lignes	Isolation directe d'une colonne/condition en une commande pandas - ainsi trouvés les 7 bugs réels
Tests automatisés	Difficile à intégrer nativement	Chaque étape est une fonction testable unitairement (32 tests écrits)
Installation	Lourde (JDK, IDE ~1-2 Go)	Légère (<code>pip install</code>)
Connecteurs	Nombreux, sans code	Couverture native CSV/Excel/PostgreSQL via pandas/SQLAlchemy
Approche visuelle	Flux visible d'un coup d'œil	Flux linéaire dans le code, documenté par un diagramme et ce rapport

Table 4.1 : Comparaison Talend Open Studio vs pipeline Python

Choix retenu : Python. Le facteur décisif a été la traçabilité Git pour un travail d'équipe à cinq avec relecture de code, et la possibilité d'écrire des tests automatisés qui ont concrètement révélé 3 des 7 bugs corrigés. L'inconvénient assumé - moins immédiatement lisible pour un public non technique en soutenance - est compensé par un diagramme du schéma et la documentation écrite du pipeline.

Chapitre 5

Analyses BI et tableaux de bord Power BI

Conformément au cahier des charges, le rapport Power BI est construit en trois pages minimum, connectées directement à l'entrepôt de données PostgreSQL (Microsoft, 2026) : une vue d'ensemble, une analyse du profil client, et une analyse du risque financier associé au churn.

5.1 Vue d'ensemble

La page d'accueil présente les indicateurs clés : taux de churn global (41,16%, cohérent avec le taux pondéré par événement calculé dans le pipeline ETL), nombre de clients distincts (319K, cohérent avec `dim_client`), taux de rétention (58,84%), et la Value-at-Risk (VaR) associée au churn (5,42 milliards de TND). Un graphique combiné retrace l'évolution annuelle du nombre de churns et sa croissance année sur année.

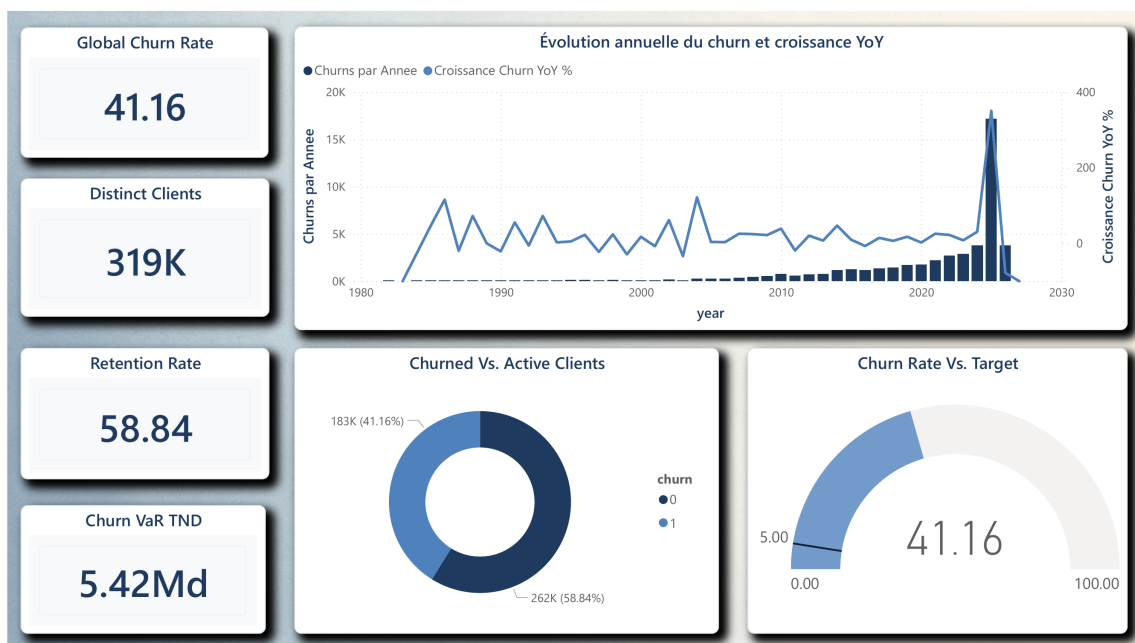


Figure 5.1 : Vue d'ensemble - taux de churn global, rétention, évolution annuelle et VaR

5.2 Analyse du profil client

La deuxième page décompose le taux de churn par profil : âge, statut marital, score KYC et segment (PARTYCLASS), avec des filtres interactifs permettant de croiser ces dimensions. Le graphique salaire moyen churné vs actif fait apparaître un écart marqué - les clients churnés ayant en moyenne un salaire inférieur aux clients actifs sur le segment observé.

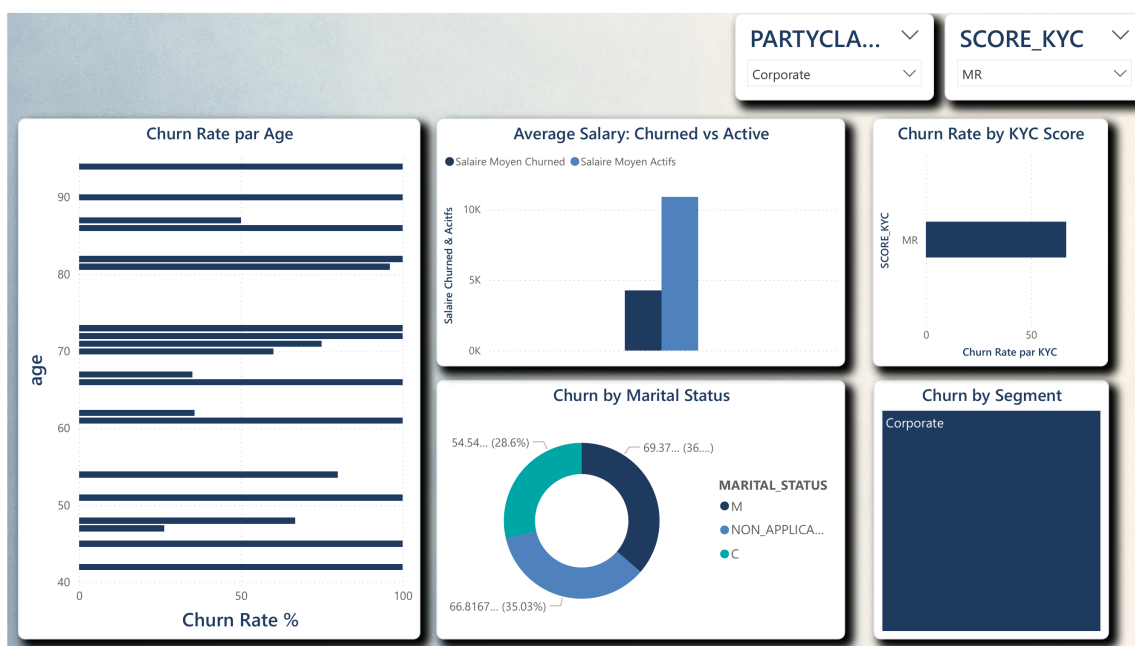


Figure 5.2 : Analyse du churn par profil client (âge, statut marital, score KYC, segment)

5.3 Analyse produits et risque financier

La troisième page explore la relation entre le nombre de produits détenus et le taux de churn (une courbe en cloche, le churn augmentant fortement puis se stabilisant à un plateau proche de 100% au-delà d'un certain seuil de produits), ainsi que la répartition churné/actif par ligne de produit - les dépôts (DEPOSITS) affichant le taux de churn le plus élevé (95,71%), cohérent avec leur nature de produits à échéance.



Figure 5.3 : Analyse du risque par produit - solde, durée avant churn, répartition par ligne de produit

5.4 Décomposition de la Value-at-Risk (VaR)

Une page complémentaire décompose la VaR totale liée au churn (54,08 millions de TND sur le périmètre filtré) par segment (PARTYCLASS), ligne de produit et score KYC, à travers une visualisation en flux (*decomposition tree*) permettant à un analyste métier d’explorer librement les contributeurs au risque. Une matrice de risque (segment × score KYC) complète cette analyse avec un score de risque brut par cellule.

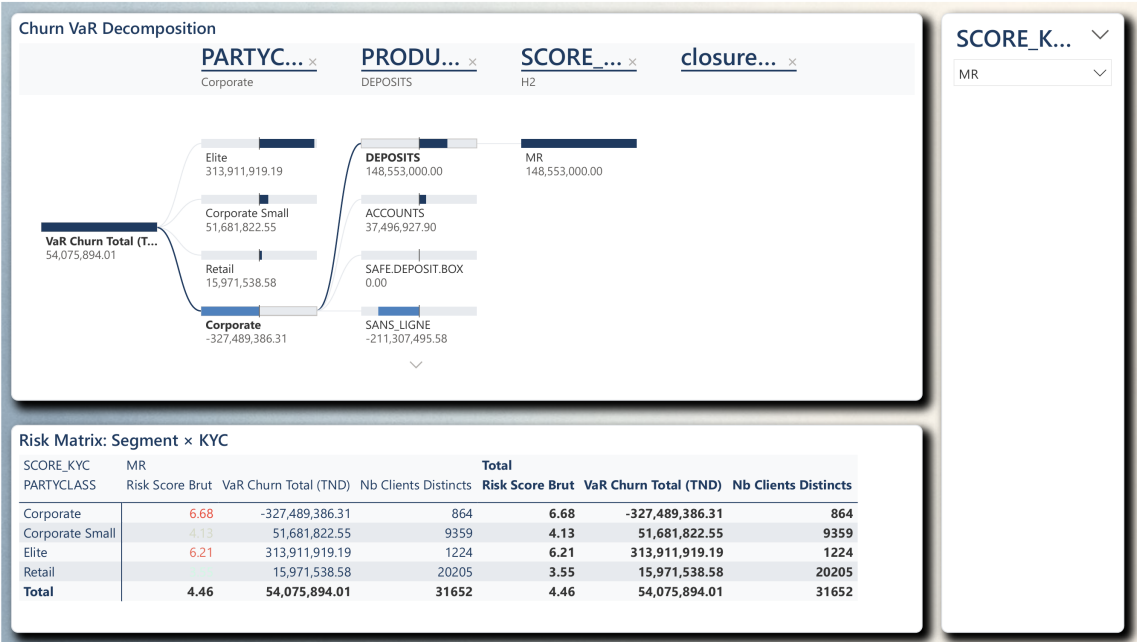


Figure 5.4 : Décomposition de la Value-at-Risk du churn par segment, produit et score KYC

5.5 Limites de cette partie

La décomposition de la VaR fait apparaître des valeurs négatives pour certaines branches (ex. segment **Corporate** : -327,5 millions de TND), signalant que la mesure DAX sous-jacente combine des flux de signe opposé (probablement solde net positif et négatif agrégés) - un point à clarifier avec l’équipe avant toute utilisation décisionnelle de cet indicateur en l’état.

Chapitre 6

Modélisation supervisée - prédiction du churn

6.1 Grain et périmètre

Le modèle supervisé est entraîné au grain **client** (`client_key`), avec la définition du churn agrégée par `min` (churn = 1 si et seulement si tous les comptes du client sont fermés — voir section 2). La table de faits est agrégée par client avant tout entraînement, ramenant le jeu de données à **319 129 lignes**, pour un taux de churn de **36%** — un déséquilibre de classes modéré plutôt qu'extrême.

6.2 Sélection des variables

Chaque variable exclue l'a été pour une raison documentée individuellement dans le notebook :

- `client_key` — identifiant, sans valeur prédictive.
- `acct_balance` — **fuite d'information** (voir section 6.3).
- `nb_comptes` — redondante avec `nb_produits` (99,1% de correspondance entre les deux colonnes).
- `NATIONALITY`, `RESIDENCE` — quasi constantes (majoritairement "TN").

Le jeu de variables retenu compte 6 variables numériques (`salary`, `amount_total`, `fixedrate_mean`, `nb_produits`, `age`, `salary_missing`) et 8 variables catégorielles (`MARITAL_STATUS`, `NATURE_CLIENT`, `PARTYCLASS`, `LOB`, `INDUSTRY`, `SCORE_KYC`, `COMPLETED_FILE`, `BRANCH`).

6.3 Fuite de données détectée et corrigée

La vérification systématique du taux de churn selon qu'une variable est renseignée ou non (méthode appliquée à chaque colonne présentant des valeurs manquantes) a révélé un cas de fuite évident : parmi les clients pour lesquels `acct_balance` est manquant, **100%** ont churné. Une variable parfaitement corrélée à l'absence de donnée pour la classe positive n'est pas un signal métier exploitable en production (elle encoderait, de fait, le label lui-même) — `acct_balance` a donc été exclue des variables explicatives.

À l'inverse, la même vérification sur `salary` n'a pas révélé de fuite : le taux de churn est proche, que le salaire soit renseigné (35,57%) ou manquant (37,69%). `salary_missing` a donc été conservé comme indicateur binaire légitime, et `salary` lui-même imputé par la médiane plutôt que laissé manquant.

Une vérification similaire sur `COMPLETED_FILE` (dossier client complet ou non) confirme un signal légitime et fort : 49,6% de churn parmi les dossiers incomplets contre 22,5% parmi les dossiers complets — une corrélation plausible d'un point de vue métier (un dossier incomplet signale un

engagement client plus faible), retenue comme variable explicative.

Cette démarche — vérifier systématiquement le taux de churn conditionnel à la présence/absence de chaque variable avant de l'utiliser — est la même discipline que celle qui avait permis de détecter une fuite liée à l'ancienneté du compte lors d'une itération antérieure du modèle au grain compte : elle est désormais systématique dans le pipeline de modélisation de l'équipe.

6.4 Prétraitement

Les variables catégorielles à faible cardinalité (MARITAL_STATUS, NATURE_CLIENT, PARTYCLASS, SCORE_KYC, COMPLETED_FILE, LOB) sont encodées en *one-hot* ; BRANCH (141 valeurs) et INDUSTRY (637 valeurs) sont encodées par fréquence, ajustée uniquement sur l'ensemble d'entraînement pour éviter toute fuite. Le jeu de données est séparé 80/20 de manière stratifiée, et SMOTE (Chawla et al., 2002) est appliqué uniquement sur l'ensemble d'entraînement pour rééquilibrer les classes — chaque modèle est testé à la fois avec et sans SMOTE, soit dix configurations au total.

6.5 Modèles comparés et résultats

Six modèles ont été entraînés : régression logistique, K plus proches voisins (KNN), arbre de décision, forêt aléatoire (Breiman, 2001), machine à vecteurs de support (SVM) et XGBoost (Chen and Guestrin, 2016), chacun avec et sans SMOTE.

Modèle	Precision	Recall	F1-score	ROC-AUC	Temps (s)
XGBoost (avec SMOTE)	91,82%	84,92%	88,24%	94,64%	2,84
XGBoost (sans SMOTE)	92,23%	84,48%	88,19%	94,74%	1,97
Random Forest (avec SMOTE)	91,67%	84,83%	88,12%	94,37%	24,08
Random Forest (sans SMOTE)	92,35%	84,30%	88,14%	94,40%	16,15
Arbre de décision (sans SMOTE)	92,17%	84,14%	87,97%	94,16%	1,22
KNN (avec SMOTE)	55,20%	68,83%	61,27%	74,30%	1,23
SVM (avec SMOTE, sous-éch.)	53,99%	67,77%	60,10%	71,04%	437,22
KNN (sans SMOTE)	62,98%	53,11%	57,63%	75,01%	0,06
Arbre de décision (avec SMOTE)	37,41%	98,19%	54,17%	49,77%	1,76
SVM (sans SMOTE, sous-éch.)	60,00%	47,89%	53,27%	70,57%	115,38

Table 6.1 : Comparaison des modèles de classification (grain client, 10 configurations avec/sans SMOTE)

L'analyse d'importance des variables d'XGBoost identifie `amount_total` comme largement dominante (52,7% du poids de décision), loin devant `nb_produits` (7,7%), `COMPLETED_FILE = YES` (6,1%), `LOB = 30` (4,7%) et `fixedrate_mean` (3,5%).

6.6 Critère de sélection : pourquoi le recall prime sur ce problème

Le F1-score et le ROC-AUC restent de bons indicateurs globaux, mais le critère décisif retenu par l'équipe pour arbitrer entre les modèles les plus performants a été le **recall** (rappel) de la classe churn, pour une raison métier précise : dans un contexte de rétention bancaire, le coût d'un **faux négatif** (un client qui va réellement churner mais que le modèle classe comme fidèle) est structurellement plus élevé que celui d'un **faux positif** (un client contacté inutilement par un conseiller). Ne pas détecter un churn réel signifie perdre le client sans avoir tenté la moindre action de rétention ; contacter à tort un client fidèle coûte au pire un appel superflu.

Sur ce critère, **XGBoost avec SMOTE obtient le meilleur recall parmi les modèles dont les autres métriques restent exploitables** (84,92%), devant Random Forest avec SMOTE (84,83%) et XGBoost sans SMOTE (84,48%). L'écart avec ces deux challengers est faible en valeur absolue, mais XGBoost avec SMOTE l'emporte également sur le F1-score (88,24%, le meilleur du comparatif) et reste très rapide à entraîner (2,84 secondes) — un compromis recall/précision/temps d'exécution qui n'est dominé sur aucun axe par un autre modèle non dégénéré.

Point de vigilance méthodologique : le recall le plus élevé du comparatif appartient en réalité à l'arbre de décision avec SMOTE (98,19%) — mais ce chiffre est trompeur. Sa précision s'effondre à 37,41% et son ROC-AUC tombe à 49,77%, soit à peine mieux que le hasard : ce modèle maximise le recall en prédisant le churn de manière quasi systématique, ce qui n'a aucune valeur opérationnelle (une équipe de rétention ne peut pas contacter la quasi-totalité de la base client). Ce résultat illustre pourquoi le recall doit toujours être lu conjointement avec la précision et le ROC-AUC, jamais isolément.

Modèle retenu : XGBoost avec SMOTE. Il maximise le recall parmi les modèles dont la précision reste exploitable, tout en offrant le meilleur F1-score et un temps d'entraînement compatible avec un réentraînement régulier en production.

6.7 Limitations assumées

KNN et SVM sont entraînés/évalués sur un sous-échantillon (coût de prédiction et noyau RBF trop élevés sur 319 129 clients dans le temps imparti), ce qui limite la comparabilité stricte de leurs résultats avec les quatre autres modèles, entraînés sur le jeu complet. Cette limitation est documentée explicitement dans le notebook.

Chapitre 7

Analyse non supervisée - segmentation et anomalies

7.1 Motivation et grain

La modélisation supervisée (chapitre 6) répond à une question binaire : *ce compte va-t-il cherner ?*. Elle ne dit rien, en revanche, sur les profils clients sous-jacents ni sur l'existence de comportements atypiques ne correspondant à aucun segment standard. Une analyse complémentaire non supervisée a donc été menée, à un grain différent : le **client** (et non le compte), avec la définition du churn agrégée par **min** (churn = 1 si et seulement si tous les comptes du client sont fermés - voir section 2). Ce grain regroupe **319 129 clients**.

Ce grain est désormais le même que celui utilisé au chapitre 6 pour la modélisation supervisée : les deux analyses partagent la même population et la même définition du churn, mais répondent à des questions différentes et complémentaires. La modélisation supervisée répond à *qui* va probablement cherner (une prédiction individuelle, exploitée par l'application de scoring du chapitre 8). L'analyse non supervisée ci-dessous répond à *quels profils* coexistent dans la base et *quels clients* s'écartent de tout profil standard — une lecture structurelle que la seule prédiction binaire ne fournit pas.

7.2 Analyse en composantes principales (ACP)

Les variables quantitatives (**salary**, **amount_total**, **fixedrate_mean**, **nb_produits**, **age**) sont standardisées puis projetées par ACP (Jolliffe, 2002), après transformation logarithmique de **salary** et transformation *symlog* de **amount_total** (positif et négatif possibles, avec des valeurs extrêmes). L'ACP retient 5 composantes ; les deux premières expliquent **54,46%** de la variance totale (34,37% + 20,09%), les trois premières **74,02%**.

7.3 Segmentation par K-means

Un algorithme K-means à $k = 4$ est appliqué sur les variables standardisées. Les quatre segments obtenus sont fortement déséquilibrés en taille :

Cluster	Taille	Profil moyen
0	125 603	Salaire moyen (729 TND), peu de produits (1,31), âge élevé (62,6 ans)
1	22 615	Salaire élevé (5 098 TND), 3,52 produits, taux fixe moyen élevé (7,44)
2	12	Cas extrêmes : amount_total moyen de 38 milliards, 744 produits en moyenne
3	170 899	Profil le plus jeune (36,3 ans), peu de produits (1,13), faible engagement

Table 7.1 : Segments K-means ($k = 4$, grain client, $n = 319\,129$)

Le cluster 2 (12 clients) correspond à des valeurs extrêmes plutôt qu'à un véritable segment métier

- à rapprocher de la détection d'anomalies ci-dessous plutôt qu'à interpréter comme une stratégie de rétention différenciée.

7.4 Détection d'anomalies par Isolation Forest

Un modèle Isolation Forest (Liu et al., 2008) est appliqué sur les mêmes variables standardisées pour identifier des clients dont le profil s'écarte significativement de la distribution générale, indépendamment de leur appartenance à un cluster K-means. **3 192 clients** (soit environ 1% de la base) sont signalés comme anomalies, dont **230** appartiennent au cluster 3 (le segment jeune à faible engagement) - suggérant que ce segment concentre une part disproportionnée de profils atypiques à investiguer manuellement (ex. combinaisons rares de solde négatif élevé et de peu de produits).

7.5 Usage envisagé

Cette analyse non supervisée complète la prédiction supervisée de deux manières : (1) elle fournit une base de segmentation exploitable pour cibler des actions marketing différenciées par profil, indépendamment du risque de churn prédit ; (2) la liste des clients anormaux peut servir de liste de priorité pour une revue manuelle par un conseiller, en complément du score de churn produit par le modèle supervisé.

Chapitre 8

Interface web et déploiement

8.1 Application Streamlit

Le modèle retenu (XGBoost avec SMOTE, chapitre 6) est restitué via une application Streamlit ([Streamlit Inc., 2026](#)) à quatre pages : un écran d'accueil présentant un résumé du jeu de données, un formulaire de scoring individuel (saisie des caractéristiques d'un client et prédiction en temps réel), un tableau de bord KPI interactif, et une page de présentation de l'équipe.

Le modèle (`model.pkl`), l'ordre des colonnes attendues (`feature_columns.pkl`) et l'encodeur de fréquence (`frequency_encoder.pkl`) sont sérialisés et chargés au démarrage de l'application, garantissant que le prétraitement appliqué en production est strictement identique à celui utilisé à l'entraînement.

8.2 Déploiement

L'application est conçue pour un déploiement sur Streamlit Community Cloud. *Lien de démonstration : à compléter par l'équipe lors du déploiement final (voir `05_web_app/README.md` du dépôt).*

8.3 Point de vigilance

Le modèle chargé est lié aux features définies au grain **client** (chapitre 6) : le formulaire de scoring doit donc saisir des caractéristiques agrégées au niveau client (ex. `nb_produits`, `amount_total` cumulé), pas des caractéristiques d'un compte isolé. Si une évolution future du notebook de modélisation change à nouveau de grain ou de jeu de variables, `model.pkl` et `feature_columns.pkl` doivent être régénérés et remplacés dans `05_web_app/models/`, faute de quoi l'application produirait des prédictions incohérentes sans erreur explicite.

Chapitre 9

Limites et perspectives

9.1 Limites connues

- **Anomalies temporelles non corrigées** (documentées mais non traitées) : 53 lignes où `DATE_OF_BIRTH` est postérieure à `CUST_OPENING_DATE`, et 114 lignes où `LAST_REVIEW_DATE` est postérieure à `NEXT_REVIEW_DATE` - l'incohérence source ne permet pas de déterminer avec certitude laquelle des deux dates est erronée.
- **Absence de validation croisée** (k-fold) sur les modèles supervisés - un seul découpage train/test a été utilisé par souci de temps d'exécution (le pipeline complet prend déjà 15 à 20 minutes).
- **Absence d'optimisation des hyperparamètres** (GridSearchCV ou équivalent) sur les modèles retenus.
- **Absence d'explicabilité par instance** (SHAP) - seule l'importance globale des variables est disponible actuellement.
- **Incohérences résiduelles entre le notebook narratif et le pipeline de production** : le notebook étape par étape (`01_etl/notbooks ETL/02_ETL.ipynb`) ne réplique pas encore la correction sur `ACCT_OPENING_DATE` appliquée dans `etl_pipeline/clean.py`, produisant un taux de nullité de `date_key` légèrement différent (22,5% contre 30,1% en production) - à harmoniser.
- **Décomposition de la VaR Power BI** présentant des valeurs négatives sur certains segments (section 5), à clarifier avant utilisation décisionnelle.

9.2 Perspectives

- Revue systématique des corrections ETL pour d'éventuels effets de bord similaires à la fuite de données décrite à la section 6.3.
- Mise en place d'une validation croisée et d'une recherche d'hyperparamètres pour affiner encore le modèle XGBoost retenu.
- Intégration de SHAP pour une explicabilité individuelle des prédictions, utile en contexte réglementaire bancaire.
- Rapprochement des résultats de la segmentation K-means (chapitre 7) avec les scores de churn supervisés, pour construire une matrice risque×valeur client actionnable par les conseillers.
- Déploiement effectif de l'application Streamlit et intégration du lien de démonstration dans la documentation du projet.

Chapitre 10

Conclusion

Ce projet a permis de construire, valider et documenter une chaîne complète d'analyse du churn bancaire, de l'exploration d'un jeu de données brut jusqu'à une application de scoring déployée. Chaque étape a fait l'objet d'une vérification concrète contre des données réelles plutôt que supposée correcte par construction : 7 anomalies ETL corrigées et documentées avec leur volumétrie exacte, une fuite de données détectée et résolue en cours de modélisation, et une analyse non supervisée complémentaire apportant un éclairage différent (segmentation, anomalies) que la seule prédiction binaire du churn ne pouvait fournir.

Le modèle XGBoost avec SMOTE retenu ($\text{recall} = 84,92\%$, $\text{F1} = 88,24\%$, $\text{AUC-ROC} = 94,64\%$) offre une base solide pour un déploiement opérationnel, sous réserve des améliorations identifiées (validation croisée, explicabilité par instance). Les tableaux de bord Power BI et l'application Streamlit complètent le dispositif en rendant les résultats accessibles à un public métier non technique, condition nécessaire à l'adoption effective d'un tel outil par les équipes de rétention client.

Équipe HAFAS - Juillet 2026

Bibliographie

- Aymen Ben Brik. Support de cours — projet intégré, m1 business analytics. Support de cours non publié, ESPRIT School of Business, 2026. Communication interne, année universitaire 2025–2026.
- Leo Breiman. Random forests. *Machine Learning*, 45(1) :5–32, 2001.
- Nitesh V. Chawla, Kevin W. Bowyer, Lawrence O. Hall, and W. Philip Kegelmeyer. SMOTE : Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16 : 321–357, 2002.
- Tianqi Chen and Carlos Guestrin. XGBoost : A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794. ACM, 2016.
- Ian T. Jolliffe. *Principal Component Analysis*. Springer, New York, 2nd edition, 2002.
- Ralph Kimball and Margy Ross. *The Data Warehouse Toolkit : The Definitive Guide to Dimensional Modeling*. Wiley, Indianapolis, IN, 3rd edition, 2013.
- Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. Isolation forest. In *2008 Eighth IEEE International Conference on Data Mining*, pages 413–422. IEEE, 2008.
- Microsoft. Power BI documentation. <https://learn.microsoft.com/power-bi/>, 2026. Consulté en 2026.
- Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn : Machine learning in Python. *Journal of Machine Learning Research*, 12 : 2825–2830, 2011.
- Streamlit Inc. Streamlit documentation. <https://docs.streamlit.io/>, 2026. Consulté en 2026.
- Talend / Qlik. Talend open studio documentation. <https://www.talend.com/>, 2026. Consulté en 2026.

Annexe A

Annexes

A.1 Glossaire

Churn Attrition, résiliation ou fermeture de la relation client.

ETL *Extract, Transform, Load* - processus d'extraction, de transformation et de chargement des données.

Schéma en étoile Modèle dimensionnel avec une table de faits centrale reliée à des tables de dimension.

SK / Surrogate Key Clé de substitution technique, indépendante de toute clé métier, utilisée pour relier les tables de l'entrepôt.

SMOTE *Synthetic Minority Over-sampling Technique*, technique de rééchantillonnage des classes minoritaires.

VaR *Value at Risk*, mesure du risque financier potentiel.

F1-score Moyenne harmonique de la précision et du rappel, métrique de référence pour une classification déséquilibrée.

ACP / PCA Analyse en Composantes Principales, technique de réduction de dimensionnalité.

A.2 Structure du dépôt de code

```
ProjetIntegre_Churn_HAFAS/  
|-- 00_documentation/      Cahier des charges, guide etudiant  
|-- 01_etl/                Pipeline ETL + notebooks EDA/ETL  
|-- 02_data_warehouse/     Schema SQL, script de chargement  
|-- 03_power_bi/           Rapport Power BI (.pbix)  
|-- 04_machine_learning/   Notebook de modelisation supervisee  
|-- 05_Datamining/         Notebook de segmentation non supervisee  
|-- 05_web_app/            Application Streamlit + modeles serialises  
|-- data/                  Jeu de donnees anonymise  
|-- test_coherence_donnees.py  Script de validation qualite (32 tests)
```

A.3 Extrait de code - contrôle de nullité structurelle de date_key

```
n_date_key_null = fact["date_key"].isna().sum()  
n_opening_date_null = fact["ACCT_OPENING_DATE"].isna().sum()  
if n_date_key_null != n_opening_date_null:  
    logger.error("INCOHERENCE : echec de jointure reel a investiguer.")  
else:  
    logger.info(  

```

```
) "date_key : nullite 100% structurelle, aucun echec de jointure."
```